SSRF — Acceso al panel administrativo interno mediante localhost

1. Título

Server-Side Request Forgery (SSRF) — Acceso a un panel administrativo interno y eliminación de un usuario

2. Descripción

Se identificó una vulnerabilidad de Server-Side Request Forgery (SSRF) en la funcionalidad de consulta de stock de una aplicación web.

La aplicación recibe una solicitud `POST` al endpoint `/product/stock`, cuyo parámetro `stockApi` contiene una URL utilizada por el servidor para consultar el servicio de inventario.

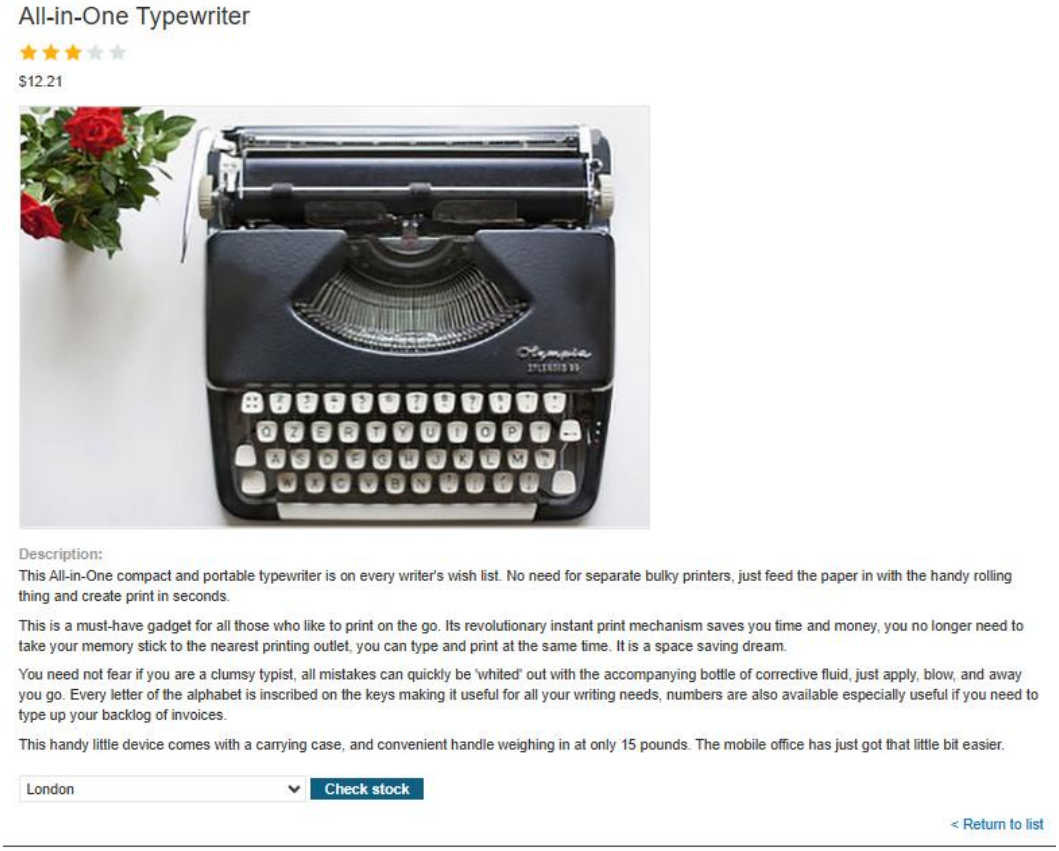
Al modificar el valor de este parámetro para apuntar a `http://127.0.0.1/admin`, fue posible acceder al panel administrativo interno desde el servidor vulnerable.

A partir de este acceso, se utilizó la funcionalidad administrativa para eliminar al usuario `carlos`, completando el objetivo del laboratorio de PortSwigger Web Security Academy.

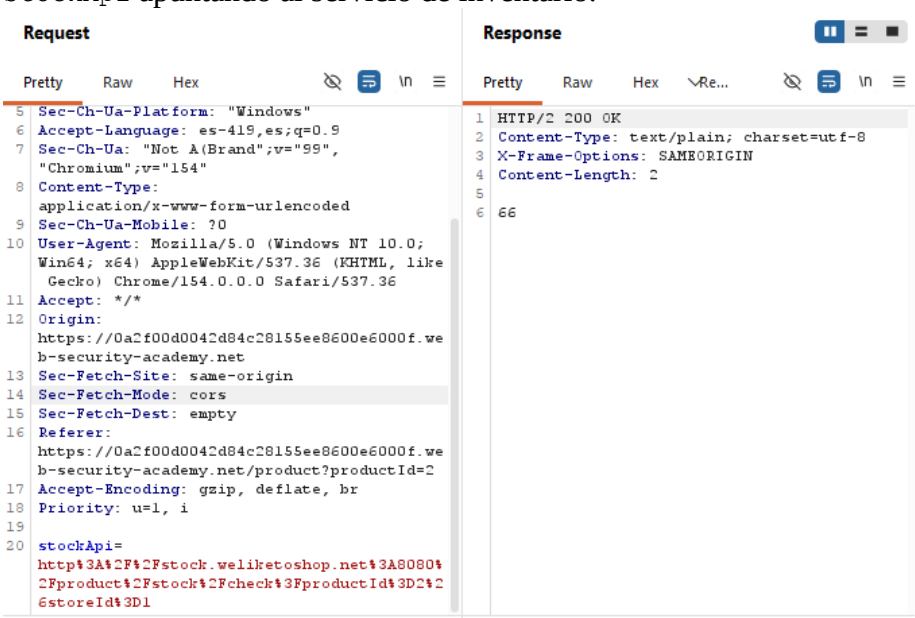
3. Evidencia

Se recopilaron seis capturas de pantalla que documentan el proceso de análisis y explotación:

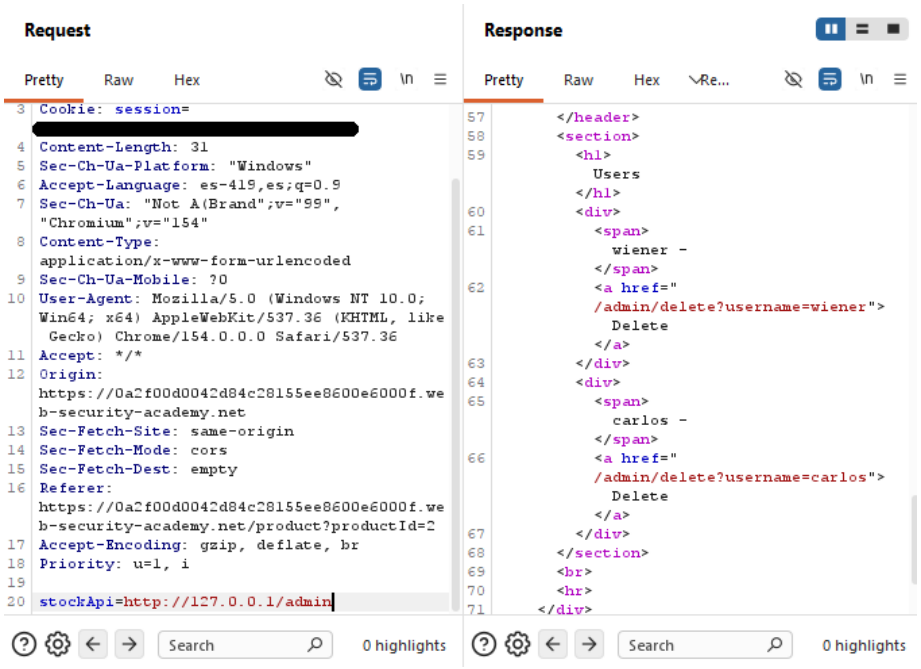
1. **Página inicial:** estado inicial de la aplicación antes de las pruebas.



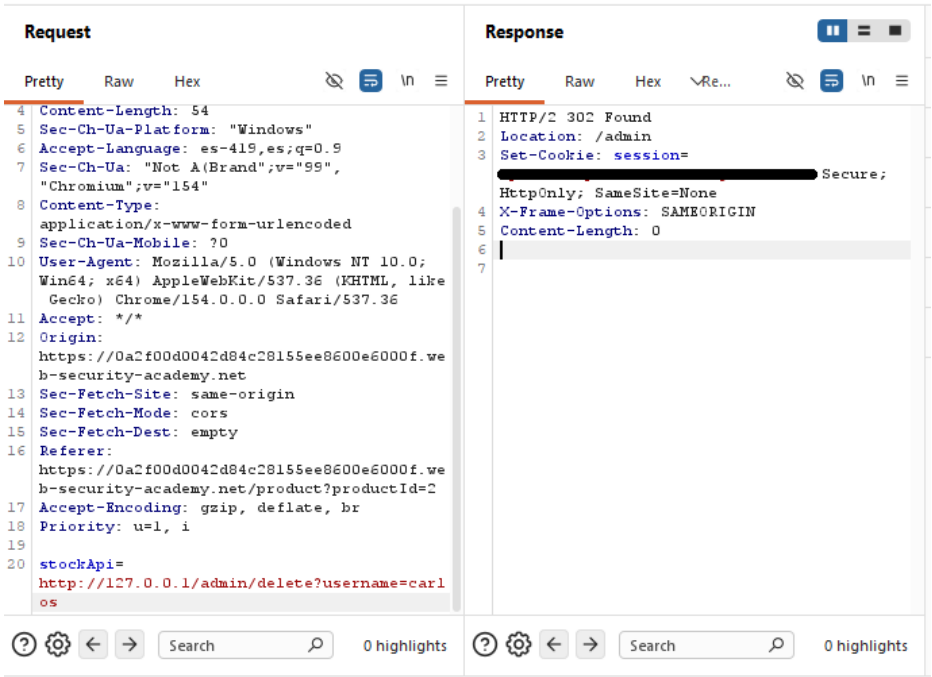
2. **Petición original de stock:** solicitud `POST /product/stock` con el parámetro `stockApi` apuntando al servicio de inventario.

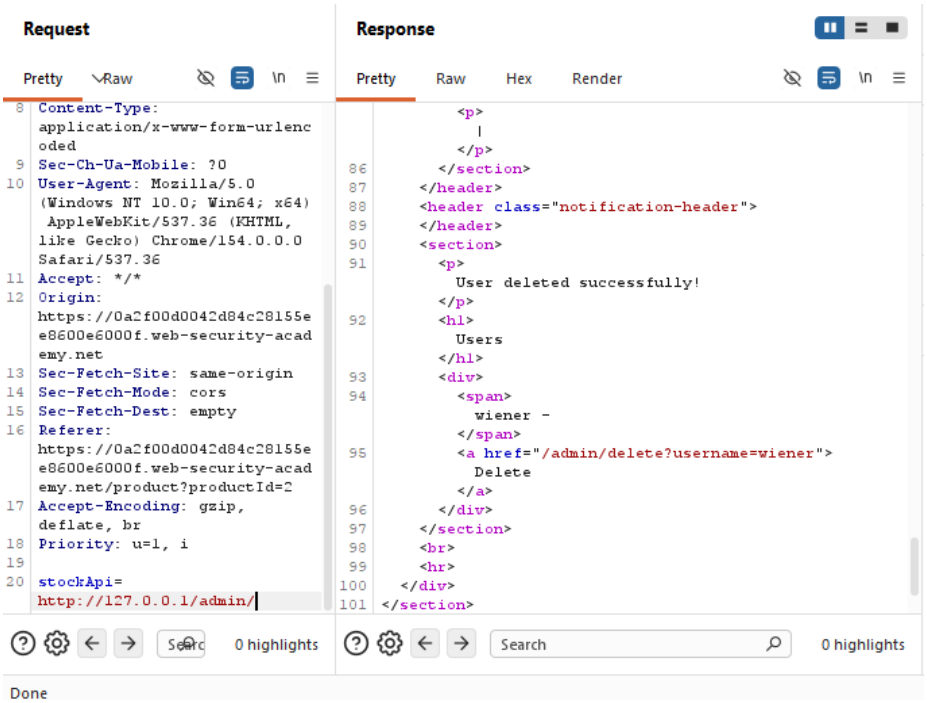


3. **Acceso al panel administrativo:** petición con `stockApi` apuntando a `http://127.0.0.1/admin`.

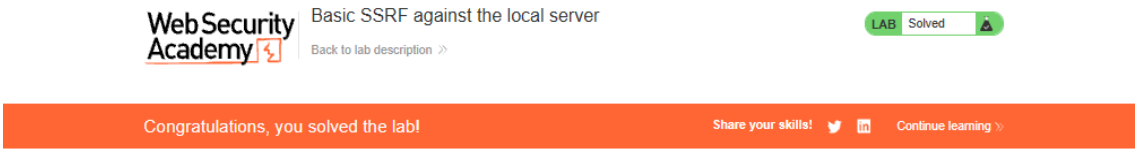


4. **Eliminación del usuario:** petición de eliminación y respuesta HTTP `302 Found`.





5. **Laboratorio resuelto:** confirmación de que el laboratorio fue completado correctamente.



La respuesta `302 Found` representa una redirección HTTP. El resultado de la acción se corroboró mediante la evidencia posterior y la finalización del laboratorio.

4. Impacto (CIA)

Confidencialidad:
La vulnerabilidad permite acceder a recursos internos que podrían exponer información administrativa o datos no disponibles públicamente.

Integridad:
Se demostró la posibilidad de modificar el estado de la aplicación mediante una acción administrativa no prevista para un usuario externo, eliminando al usuario `carlos`.

Disponibilidad:
No se comprobó una afectación directa de la disponibilidad del servicio. Sin embargo, el acceso a funciones administrativas podría permitir acciones que afecten la operación de la aplicación.

5. Causa raíz

La causa raíz es la confianza indebida en una URL proporcionada por el cliente mediante el parámetro `stockApi`.

El servidor realiza solicitudes a la URL indicada sin aplicar controles suficientes para restringir el destino. Esto permite que una funcionalidad destinada a consultar el inventario sea utilizada para acceder a recursos internos del propio servidor.

La validación insuficiente del destino y la ausencia de restricciones efectivas sobre las solicitudes salientes permiten el abuso de esta funcionalidad.

6. Remediación

- **Restringir los destinos permitidos:** aceptar únicamente servicios de inventario definidos previamente mediante una lista de permitidos (*allowlist*).
- **Evitar URLs arbitrarias:** utilizar identificadores de productos o servicios en lugar de aceptar URLs completas controladas por el cliente.
- **Validar las solicitudes salientes:** comprobar el esquema, el host, el puerto y la dirección de destino antes de realizar cada solicitud.
- **Bloquear el acceso a recursos internos:** impedir que esta funcionalidad alcance `localhost`, direcciones privadas, interfaces administrativas y servicios internos no autorizados.
- **Aplicar controles de red:** restringir las conexiones salientes mediante reglas de firewall y segmentación de red.
- **Separar privilegios:** asegurar que el servicio de inventario no pueda ejecutar operaciones administrativas y que los endpoints internos exijan autorización adecuada.

La validación de URLs debe contemplar redirecciones y resolución DNS para evitar que los controles puedan eludirse mediante destinos alternativos.

7. CVSS aproximado

CVSS v3.1: 8.1 — High (Alto), estimación provisional.

Vector orientativo: `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

La puntuación es orientativa y debe revisarse según las condiciones reales de explotación, los privilegios requeridos y el alcance del impacto. El resultado del laboratorio demuestra acceso administrativo y una modificación de datos, pero no permite establecer por sí solo la puntuación exacta para un entorno de producción.

8. Prioridad

Alta.

La vulnerabilidad permite utilizar el servidor como intermediario para alcanzar recursos internos y, en este laboratorio, acceder a una función administrativa capaz de eliminar un usuario.

Se recomienda corregir la validación del destino de `stockApi`, restringir las solicitudes salientes y revisar la exposición de los servicios internos antes de desplegar la aplicación en producción.